Location-Mismatch OTP Verification & Boundary Check

Comprehensive Technical Specification, Haversine Math, Decision Matrix & Edge Case Analysis

Platform: InnoIDE Desktop (Electron / React) | Hardware: ESP32-S3 (COM9) | Subsystem: Geofence & OTP Guard

1. Executive Summary & Core Architectural Rule

Core Question: ***Is OTP access required to turn OFF a switch/device when the location is WITHIN the boundary?***

Definitive Answer: NO. Under normal operation, **OTP access is NOT required to turn OFF if the device/user location is within the boundary** (distance ≤ threshold, default 100m). The system is designed as a **Location-Mismatch Guard**. If the current GPS location matches or is within the configured radius of the Turn-ON location, the switch turns OFF immediately without any OTP challenge.

Important Caveat: There are specific edge cases (such as missing prior location history, GPS permission denial, or stale mock timestamps) where an OTP challenge **will still be triggered** as a fail-safe security measure, even if the user is physically inside the boundary perimeter.

2. Subsystem Architecture & Key Components

The Location-Mismatch OTP and Geofence subsystem spans four interconnected application layers:

1. locationOtpService.js Core engine: Haversine distance, GPS accuracy buffer, cryptographically secure 6-digit OTP generation, 20-minute lifetime, and single-use token lifecycle management.	**2. electron/ipc/index.js** Companion Server: Serves /setSwitchStatus and /api/otp/*. Performs server-side distance verification before dispatching serial write commands to COM9.
3. useHardwareActuation.js Desktop IDE UI: Intercepts switch toggles in the App Preview Simulator, initiates OTP challenges on mismatch, and renders LocationOtpVerificationModal.jsx.	**4. companionHtmlGenerator.js** Mobile Web App: Evaluates client-side geofence rules, tracks mobile location history, and guards against unauthorized switch-off commands over cloud tunnels.

3. Haversine Distance & Dynamic GPS Accuracy Buffer

- **Haversine Formula:** Distance between previous location (lat1, lon1) and current location (lat2, lon2) is calculated using spherical trigonometry with Earth radius R = 6,371,000 meters.
- **Dynamic GPS Accuracy Buffer:** To prevent false alarms caused by satellite jitter, the effective threshold is dynamically expanded: effectiveThreshold = thresholdMeters + accuracyBuffer, where accuracyBuffer = min(max(acc_prev, acc_cur), 50).

```
// locationOtpService.js (Lines 107-113)
const prevAcc = Number(prevLoc.accuracy || 0);
const curAcc = Number(curLoc.accuracy || 0);
const accuracyBuffer = Math.min(Math.max(prevAcc, curAcc), 50);
const effectiveThreshold = thresholdMeters + accuracyBuffer; // Default: 100m + buffer

const isMismatch = rawDist > effectiveThreshold;
// Result: If rawDist <= effectiveThreshold (WITHIN BOUNDARY) -> isMismatch = FALSE
```

4. Comprehensive Decision Matrix (Switch Operations vs. Location State)

The following matrix outlines all operational conditions, whether an OTP challenge is enforced, and the resulting hardware actuation:

Switch Action	Current Location	Prior Baseline State	OTP Required?	System Action & Hardware Result
Turn ON	Any coordinate	Any state	**NO**	Immediate actuation (LED:1\r\n). Coordinates recorded as baseline for subsequent OFF operations.
Turn OFF	**Within Boundary** (dist ≤ 100m)	Valid recorded ON location	**NO**	Immediate actuation (LED:0\r\n). Normal operation within permitted perimeter. No OTP challenge.
Turn OFF	**Outside Boundary** (dist > 100m)	Valid recorded ON location	**YES**	Location mismatch detected. Blocks hardware command until valid 6-digit OTP is verified.
Turn OFF	Within Boundary	**No prior location** (!prevOperationLoc)	**YES** (Web App)	Web companion fallback guard: cannot compute displacement, so enforces OTP verification before first switch-off.
Turn OFF	**GPS Unavailable** (Permission denied)	Valid recorded ON location	**YES**	MISSING_CURRENT_LOCATION safety fallback. Cannot verify proximity, so prompts for OTP verification.
Turn OFF	**Invalid Coords** (NaN / Malformed)	Valid recorded ON location	**YES**	INVALID_COORDINATES fallback. Distance calculation yields null, requiring OTP verification.

5. Deep Dive: When OTP is Triggered Despite Being Within Boundary

Developers and QA engineers should take note of the four specific code pathways where an OTP prompt appears even when the operator is physically co-located with the ESP32 hardware:

Edge Case 1: First Switch-OFF in Web Companion (!prevOperationLoc)
In companionHtmlGenerator.js (lines 2457–2462), the rule checks: var locationMismatchExceedsThreshold = !prevOperationLoc || rawDistance == null || rawDistance > ruleThreshold;. If the companion app is launched and the operator toggles OFF directly without having first toggled ON, prevOperationLoc is null. The system enforces an OTP challenge to confirm the operator's identity.

Edge Case 2: Geolocation Permission Denied or GPS Disabled
In locationOtpService.js (lines 80–87), if curLoc is missing latitude or longitude, the function returns { isMismatch: true, reason: 'MISSING_CURRENT_LOCATION' }. The system refuses to assume proximity when geolocation telemetry is unavailable.

Edge Case 3: Stale Demo Geo-Log Sanitization (isStaleGeolog)
Both the Electron backend and mobile companion filter out mock test logs (e.g. timestamps containing 12:53:01, loc-init-1). If the previous ON location originated from a mock test suite, it is discarded, causing the baseline location to become null.

Edge Case 4: Development / Demo Mode Bypass
During testing with devDemoMode: true, the 6-digit OTP is rendered directly inside the modal with a 1-click Quick-Fill button. Additionally, fallback client tokens (vt-local-*, vt-demo-*) are automatically accepted by the backend to prevent developer lockout.

6. Cryptographic OTP & Single-Use Token Lifecycle

When an OTP challenge is required, the system executes an atomic authorization flow:

1. **Challenge Generation:** defaultLocationOtpService.createChallenge() generates a secure 6-digit numeric code with a 20-minute lifetime synchronized with the active QR validation session.
2. **Verification:** User inputs OTP via LocationOtpVerificationModal.jsx; backend executes verifyOtp(), checking attempts (max 3) and resend rate-limiting cooldown (30s).
3. **Token Issuance:** On success, a 48-character single-use cryptographic token (vt-*) is issued.
4. **Hardware Actuation & Consumption:** The client submits the token with the switch-off command. The Companion Server validates the token and calls consumeToken() immediately before writing LED:0\r\n to UART. The token is immediately deleted to prevent replay attacks.

7. Codebase Traceability Matrix

File Path	Line Range	Key Function / Handler	Architectural Responsibility
src/components/esp32/locationOtpService.js	L68–L122	isLocationMismatch()	Computes Haversine distance, applies accuracy buffer, evaluates mismatch flag
electron/ipc/index.js	L921–L956	/setSwitchStatus	Companion Server endpoint; enforces OTP before serial write on switch OFF
src/components/esp32/useHardwareActuation.js	L100–L105	liveLedToggle()	Desktop UI switch toggle handler; conditionally prompts verification modal
src/components/esp32/companionHtmlGenerator.js	L2447–L2465	sendSwitchAsync()	Mobile companion client runtime; executes client-side geofence rule check
src/components/flasherConstants.js	L36–L47	RULE_CONDITION_TYPES	Rule schema definition for 'Location Mismatch on Turn OFF (OTP Guard)'